

Computer Architecture (25VBT0C104)



Module: 04

Instruction Processing & Parallel Architectures

Module Information

Field	Details
Course Title	Computer Architecture
Course Code	25VBT0C104
Module Number	Module 4 of 5
Module Title	Instruction Processing and Parallel Architectures
Semester	I
Credits	4

Module Overview

Module 4 of Computer Architecture focuses on two pivotal themes: how a processor executes instructions, and how multiple processors can work together to solve problems faster than a single processor ever could.

The module opens with the anatomy of the basic processing unit — the internal organisation of a CPU including the Arithmetic Logic Unit, registers, control unit, and buses. You will trace the life of a single instruction through its fetch, decode, execute, and write-back phases, developing a clear mental model of the instruction execution cycle.

A significant portion of the module is dedicated to control unit design. Two philosophies — hardwired control and microprogrammed control — represent fundamentally different engineering trade-offs between speed and flexibility. Understanding both approaches is essential for evaluating real processor designs.

The module then introduces pipelining — the single most impactful performance technique in processor design. By overlapping the execution of multiple instructions (like an assembly line in a factory), pipelining dramatically increases throughput. However, dependencies between instructions create hazards that must be carefully managed. You will study structural, data, and control hazards, along with the techniques used to mitigate them.

The final section broadens the perspective from a single processor to systems of many processors working in parallel. Parallel computer models — including shared-memory multiprocessors, distributed-memory multicomputers, and theoretical models like PRAM and VLSI — provide the conceptual framework for understanding modern multi-core chips, cloud computing clusters, and high-performance computing systems.

By the end of this module, you will be equipped to apply knowledge of instruction execution to analyse processor performance, and to evaluate parallel architecture models in the context of real computing systems — including India's growing HPC infrastructure.

Learning Objectives

By the end of this module, students will be able to:

- Apply knowledge of basic processing unit design to trace instruction execution through the fetch-decode-execute cycle.
- Analyse and differentiate between hardwired and microprogrammed control unit designs.
- Apply pipelining principles to calculate throughput improvements and identify pipeline stages.
- Analyse the three categories of pipeline hazards and evaluate techniques to detect and resolve them.
- Apply knowledge of parallel computer models (multiprocessor, multicomputer, PRAM, VLSI) to classify and compare real parallel architectures.

Learning Outcomes

Upon successful completion of this module, students will be able to:

- Trace the complete instruction execution cycle (IF, ID, EX, MEM, WB) and identify the role of each functional unit.
- Compare hardwired and microprogrammed control units in terms of speed, flexibility, and implementation complexity.
- Calculate pipeline speedup and analyse the impact of pipeline stages and clock cycles on throughput.
- Identify data, structural, and control hazards in instruction sequences and apply forwarding, stalling, and branch prediction techniques.
- Classify parallel architectures using Flynn's taxonomy and evaluate the design trade-offs of multiprocessors and multicomputers.

Table of Topics

Sub-Unit	Topic	Coverage
4.1	Basic Processing Unit Design	ALU, registers, PC, IR, MAR, MDR, internal buses
4.2	Instruction Execution Steps	IF, ID, EX, MEM, WB phases; RTL description
4.3	Multiple Bus Organisation	Single, dual, triple bus structures; advantages
4.4	Control Unit Design: Hardwired	Finite state machines, decoder logic, speed advantage
4.5	Control Unit Design: Microprogrammed	Microinstructions, control memory, Wilkes model
4.6	Pipelining Basics	Pipeline stages, throughput, latency, speedup formula
4.7	Pipeline Hazards	Structural, data (RAW/WAW/WAR), control hazards
4.8	Impact on Instruction Sets	Pipeline-friendly ISA design, RISC vs CISC context
4.9	Parallel Computer Models	Flynn's taxonomy, multiprocessors, multicomputers
4.10	PRAM and VLSI Models	Theoretical parallel models, complexity, VLSI area-time trade-offs

4.1 Basic Processing Unit Design

The processing unit, or Central Processing Unit (CPU), is the brain of a computer system. Every computation, every decision, and every data movement is ultimately orchestrated by the CPU. To understand how instructions are executed, we must first understand the functional components that make up the processing unit and how they are interconnected.

4.1.1 Key Functional Components

The CPU is composed of several specialised units that work in concert:

- **Arithmetic Logic Unit (ALU):** Performs arithmetic operations (add, subtract, multiply) and logical operations (AND, OR, NOT, XOR, compare). The ALU receives operands, performs the operation, and outputs a result along with condition flags (Zero, Negative, Carry, Overflow).
- **Control Unit (CU):** The "director" of the CPU. It interprets each instruction fetched from memory and generates the control signals that direct all other components to perform the appropriate actions in the correct sequence.
- **Register File:** A small, ultra-fast set of storage locations inside the CPU. Registers hold operands for the ALU, intermediate results, and control information. Modern processors have 8 to 32 general-purpose registers.
- **Program Counter (PC):** A special-purpose register that holds the memory address of the next instruction to be fetched. After each fetch, PC is automatically incremented (or updated by a branch instruction).
- **Instruction Register (IR):** Holds the currently executing instruction after it has been fetched from memory. The control unit decodes the instruction stored in the IR.
- **Memory Address Register (MAR):** Holds the memory address to be accessed (read or written) during a memory operation.

- Memory Data Register (MDR): Temporarily holds the data read from or to be written to the memory address in MAR. Acts as a buffer between the CPU and external memory.

4.1.2 Internal Buses

The components of the CPU are connected by internal buses — shared pathways over which data travels. The width of these buses (8, 16, 32, 64 bits) determines how much data can be transferred per operation.

- Data Bus: Carries the actual data being transferred between components.
- Address Bus: Carries the memory address being accessed. The width of the address bus determines the maximum addressable memory space (32-bit address bus $\rightarrow 2^{32} = 4$ GB of addressable space).
- Control Bus: Carries control signals (read/write, interrupt, clock) that synchronise component operations.

💡 Did You Know?

India's Shakti processor project, developed at IIT Madras since 2014, is the country's first indigenous processor family. The E-class Shakti core, targeting IoT applications, has a 5-stage pipeline and 32 general-purpose registers. RISC-V based Shakti processors have been successfully taped out at ISRO's Semi-Conductor Laboratory (SCL) in Chandigarh, marking a milestone for India's semiconductor self-reliance.

4.2 Instruction Execution Steps

Every instruction that a CPU executes passes through a defined sequence of steps. This sequence is known as the instruction execution cycle (or fetch-execute cycle). Although the exact number of steps varies by processor design, most modern CPUs use a five-stage model.

The Five Stages of Instruction Execution

Stage 1: Instruction Fetch (IF)

The CPU reads the instruction from the memory location indicated by the Program Counter (PC) into the Instruction Register (IR).

- Action: $MAR \leftarrow PC$; Memory Read; $MDR \leftarrow Memory[MAR]$; $IR \leftarrow MDR$
- PC is incremented: $PC \leftarrow PC + instruction_size$
- Duration: Depends on cache hit (1–4 cycles) or miss (50+ cycles).

Stage 2: Instruction Decode (ID)

The control unit interprets the opcode (operation code) in the IR to determine what operation to perform. It also identifies the source operands (registers or memory addresses) and the destination.

- The instruction format is decoded: opcode, source registers ($Rs1$, $Rs2$), destination register (Rd), immediate values, or memory addresses.
- Operand values are read from the register file in parallel with decoding (register fetch).

Stage 3: Execute (EX)

The ALU performs the operation specified by the instruction. For arithmetic/logical instructions, operands are fed into the ALU and the result is computed. For memory instructions, the effective address is calculated. For branch instructions, the branch condition is evaluated.

- ADD $R1$, $R2$, $R3$: ALU computes $R2 + R3$; result held in a temporary register.

- `LOAD R1, 100(R2)`: ALU computes effective address = $R2 + 100$.
- `BEQ R1, R2, offset`: ALU checks $R1 - R2 == 0$; computes branch target address.

Stage 4: Memory Access (MEM)

This stage is only active for load and store instructions. For loads, data is read from memory at the computed effective address. For stores, data is written to memory. For non-memory instructions, this stage passes the ALU result through unchanged.

Stage 5: Write Back (WB)

The result (from the ALU or from memory for a load) is written back into the destination register in the register file, completing the instruction.

Stage	Name	Key Action
IF	Instruction Fetch	Fetch instruction from memory into IR; increment PC.
ID	Instruction Decode	Decode opcode; read source registers from register file.
EX	Execute	ALU performs operation; compute effective address for memory ops.
MEM	Memory Access	Read data from memory (load) or write data to memory (store).
WB	Write Back	Write ALU result or loaded data into destination register.

🔍 Analysing Misconceptions: Instruction Execution and Clock Cycles

✗ MYTH: Each instruction takes exactly one clock cycle to execute on a modern processor.

✓ REALITY: The time to execute an instruction varies widely. Without pipelining, a single instruction may take 5 or more clock cycles (one per stage). With pipelining, the throughput approaches one instruction per cycle, but individual instruction latency (from fetch to write-back) is still multiple cycles. Complex instructions (multiply, divide, floating-point) take even more cycles in the execute stage.

4.3 Multiple Bus Organisation

The internal organisation of a processor — specifically how its components are connected — significantly affects performance. Bus organisation determines how many data transfers can occur simultaneously within a clock cycle.

4.3.1 Single Bus Organisation

All components (ALU, registers, memory interface) share a single internal bus. Only one transfer can occur at a time.

- Advantage: Simple design, low hardware cost, easy to control.
- Disadvantage: The bus becomes a bottleneck. Most operations require 3+ bus cycles (fetch operand A, fetch operand B, store result), serialising what could be parallel operations.
- Example operation: ADD R1, R2, R3 requires: Bus transfer R2 → Temp1, then R3 → Temp2, then ALU result → R1 — three sequential bus cycles.

4.3.2 Two-Bus (Dual Bus) Organisation

Two separate buses — a source bus and a destination bus — allow two transfers to occur simultaneously. One operand can be read from the register file while the previous result is being written back.

- Advantage: Doubles potential data throughput; enables some instruction stages to overlap.
- Disadvantage: More complex control logic; additional wiring.

4.3.3 Three-Bus Organisation

Three buses allow both source operands to be read and the result to be written in a single clock cycle. This is the basis of modern high-performance register files.

- Bus A: Carries first source operand (Rs1) to the ALU.
- Bus B: Carries second source operand (Rs2) to the ALU.
- Bus C: Carries the ALU result back to the destination register (Rd).
- Advantage: ADD R1, R2, R3 completes in a single bus cycle — both operands read and result written simultaneously.
- Used in: ARM Cortex-A series, RISC-V implementations, Intel Core architecture.

💡 Did You Know?

Modern processors like the AMD Ryzen series use a "register file crossbar" — essentially a multi-port register file that can handle 4–6 reads and 2–3 writes per clock cycle, far exceeding the three-bus model. This is achieved through specialised SRAM arrays with multiple read and write ports, enabling out-of-order processors to issue multiple independent instructions simultaneously.

4.4 Control Unit Design: Hardwired Control

The control unit is responsible for generating the precise sequence of control signals that orchestrate every component of the CPU during instruction execution. Two fundamentally different approaches exist for implementing the control unit: hardwired and microprogrammed. This sub-unit examines hardwired control.

4.4.1 Concept and Operation

In hardwired control, the control unit is implemented as a fixed digital logic circuit — a combinational or sequential network of logic gates. The inputs to this circuit are the instruction opcode, condition flags, and the current state of the processor. The outputs are the control signals sent to all CPU components.

The circuit is typically designed as a Finite State Machine (FSM):

- Each state corresponds to a step in the instruction execution cycle (IF, ID, EX, MEM, WB).
- State transitions are triggered by the clock and by the decoded instruction.
- Each state produces a specific set of control signals (e.g., $ALU_op = ADD$, $RegWrite = 1$, $MemRead = 0$).

4.4.2 Design Approach

The design process for a hardwired control unit involves:

- Step 1: Define the instruction set and identify all instructions.
- Step 2: For each instruction, list the control signals required at each execution step.
- Step 3: Construct a control signal table (instruction $\times$ step $\rightarrow$ control signals).
- Step 4: Derive Boolean expressions for each control signal using Karnaugh maps or algorithmic minimisation.
- Step 5: Implement the minimised logic using combinational gates and flip-flops.

4.4.3 Advantages and Disadvantages

Advantages	Disadvantages
Very fast — signals generated directly by logic gates without memory lookup.	Rigid — adding new instructions requires redesigning the entire circuit.
Optimal for simple, fixed instruction sets (RISC architectures).	Complex to design and verify for large instruction sets (CISC).
Lower power consumption than microprogrammed control for equivalent designs.	Debugging and modification are difficult once fabricated.

Real-world use: The ARM Cortex-M0/M0+ series (widely used in embedded systems, including Tata-owned Jaguar Land Rover infotainment controllers and Indian Railways signal processing systems) uses hardwired control, enabling a tiny die area and sub-milliwatt power consumption.

4.5 Control Unit Design: Microprogrammed Control

Microprogrammed control, proposed by Maurice Wilkes in 1951, represents a more flexible approach to control unit design. Instead of implementing control logic as fixed hardware gates, control signals are stored as data in a special memory called the control store (or microprogram memory).

4.5.1 Key Concepts

- **Microinstruction:** A single row in the control store. Each bit (or field) of the microinstruction corresponds to a control signal or a group of signals. When a microinstruction is executed, its bits directly drive the control signals of the CPU.
- **Microprogram:** A sequence of microinstructions stored in the control store that collectively implement one machine instruction. Executing an ADD instruction, for example, involves executing a microprogram of 5–10 microinstructions.
- **Control Address Register (CAR):** Holds the address of the current microinstruction in the control store (analogous to the PC for machine instructions).
- **Control Data Register (CDR):** Holds the microinstruction currently being executed (analogous to the IR).
- **Control Store:** A fast read-only (or sometimes writable) memory that holds all microprograms for all machine instructions.

4.5.2 Operation

When a machine instruction is fetched:

- **Step 1:** The opcode is used to index into a mapping ROM that provides the starting address of the corresponding microprogram in the control store.
- **Step 2:** The CAR is loaded with this starting address.
- **Step 3:** Microinstructions are fetched from the control store sequentially (or with branching) and loaded into the CDR.

- Step 4: The bits of the CDR directly control CPU operations — gate enables, ALU operation codes, register selects, etc.
- Step 5: The microprogram ends with a microinstruction that initiates the fetch of the next machine instruction.

4.5.3 Wilkes Model

Maurice Wilkes' original model used two memories: a main control memory and a second memory to hold conditions for microinstruction branching. While simple, this architecture established the fundamental principle that machine instructions could be implemented by a lower-level program — a revolutionary idea in 1951.

4.5.4 Comparison: Hardwired vs Microprogrammed

Feature	Hardwired	Microprogrammed
Speed	Faster	Slower (memory lookup per cycle)
Flexibility	Rigid — hardware must change	High — change control store content
Instruction Set Size	Suited for small ISA (RISC)	Suited for large ISA (CISC)
Design Complexity	High for complex ISA	More manageable; structured
Examples	ARM Cortex-M, RISC-V	Intel x86 (pre-1995), IBM System/360

🔍 Analysing Misconceptions: Microprogrammed Control and Modern Processors

✗ MYTH: All modern processors use microprogrammed control because it's more flexible and easier to update.

✓ REALITY: Modern high-performance processors like ARM Cortex-A and RISC-V cores predominantly use hardwired control for speed. However, complex x86 processors (Intel Core, AMD Ryzen) use a hybrid approach: a hardwired "fast path" for the most common instructions and a microprogrammed "slow path" for rare, complex instructions. This hybrid gives them both performance and compatibility with the vast x86 instruction set.

✓ Let's Check

Q.No	Type	Question
1	MCQ	Which register holds the address of the next instruction to be fetched? (A) IR (B) MAR (C) PC (D) MDR
2	MCQ	In microprogrammed control, individual control signals are stored as data in: (A) The register file (B) Main memory (C) The control store (D) The instruction register
3	Fill in Blank	In a three-bus organisation, Bus A and Bus B carry _____ operands to the ALU, while Bus C carries the result back.
4	Fill in Blank	The five stages of instruction execution in order are: IF, ID, _____, MEM, WB.

5	True/False	Hardwired control is generally faster than microprogrammed control for a given instruction. (True/False)
6	True/False	The MEM stage is active for all types of instructions. (True/False)

Answers:

Q.No	Answer
1	(C) PC — the Program Counter holds the address of the next instruction.
2	(C) The control store — a special ROM/RAM containing microinstructions.
3	Source (two)
4	EX (Execute)
5	True
6	False — The MEM stage is only active for load and store instructions. For arithmetic/logical instructions, this stage is effectively bypassed.

4.6 Pipelining Basics

Pipelining is one of the most powerful techniques in computer architecture for improving processor throughput. Inspired by the concept of an assembly line in manufacturing — think of a car assembly plant where different cars are in different stages of assembly simultaneously — pipelining allows multiple instructions to be in different stages of execution at the same time.

4.6.1 The Concept

Without pipelining (sequential execution), the processor completes all stages of instruction I_1 before starting instruction I_2 . If each stage takes 1 ns and there are 5 stages, each instruction takes 5 ns. For 4 instructions: $4 \times 5 = 20$ ns total.

With pipelining, as soon as instruction I_1 moves from Stage 1 (IF) to Stage 2 (ID), instruction I_2 can enter Stage 1 (IF). Multiple instructions overlap:

Instruction	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6	Cycle 7+
I_1	IF	ID	EX	MEM	WB	—	—
I_2	—	IF	ID	EX	MEM	WB	—
I_3	—	—	IF	ID	EX	MEM	WB

With pipelining, instruction I_1 completes at cycle 5, I_2 at cycle 6, I_3 at cycle 7 — one instruction completing per cycle after the pipeline is "filled". Total for 4 instructions: 8 cycles (vs 20 without pipelining).

4.6.2 Pipeline Performance Metrics

Let k = number of pipeline stages, n = number of instructions, t = time per stage (clock period).

- Time without pipeline: $n \times k \times t$
- Time with pipeline (ideal): $(k + n - 1) \times t$
- Speedup = $(n \times k \times t) / ((k + n - 1) \times t) = n \times k / (k + n - 1)$
- As $n \rightarrow \infty$ (many instructions): Speedup $\rightarrow k$. An ideal k -stage pipeline approaches k -fold speedup.

Example: 5-stage pipeline ($k=5$), 100 instructions ($n=100$), 1 ns per stage.

- Without pipeline: $100 \times 5 \times 1 = 500$ ns
- With pipeline: $(5 + 100 - 1) \times 1 = 104$ ns
- Speedup: $500 / 104 \approx 4.81\times$ (approaching $5\times$ for large n)

4.6.3 Pipeline Latency vs Throughput

- Latency: Time for a single instruction to complete — unchanged by pipelining (still k stages $\times t$ per stage = $k \times t$). Pipelining does NOT reduce latency.
- Throughput: Number of instructions completed per unit time. Pipelining dramatically increases throughput (approaching 1 instruction per clock cycle).

💡 Did You Know?

The Intel Pentium 4 "Prescott" processor (2004) had a 31-stage pipeline — one of the deepest ever built. This allowed very high clock frequencies (3.8 GHz) but suffered from severe branch misprediction penalties (31 wasted cycles per misprediction). This is why modern Intel processors use shorter pipelines (14–19 stages) that balance frequency against hazard penalties.

4.7 Pipeline Hazards

The ideal pipeline completes one instruction every clock cycle. In practice, dependencies and resource conflicts cause the pipeline to stall — introducing "bubbles" (wasted cycles) that reduce throughput. These problems are called pipeline hazards. There are three categories of hazards.

4.7.1 Structural Hazards

A structural hazard occurs when two instructions in different pipeline stages require the same hardware resource simultaneously, and the hardware cannot accommodate both requests at once.

Example: In a simple processor with a single memory unit, instruction I_1 in the MEM stage reads from memory while instruction I_4 simultaneously tries to perform Instruction Fetch (IF) — both need memory access.

- Solution 1: Stall the pipeline — insert a "bubble" (no-operation cycle) to delay one instruction.
- Solution 2: Use separate instruction cache and data cache (Harvard architecture) — most modern processors take this approach.

4.7.2 Data Hazards

A data hazard occurs when an instruction depends on the result of a previous instruction that has not yet completed. There are three types:

RAW (Read After Write) — True Dependency

The most common hazard. Instruction I_2 tries to read a register that I_1 has not yet written back.

Example: `ADD R1, R2, R3` (I_1 writes R1 in WB stage)

`SUB R4, R1, R5` (I_2 reads R1 in ID stage — but I_1 has not written it yet!)

- Solution 1 — Stalling: Insert "bubble" cycles between I_1 and I_2 until the data is available.

- Solution 2 — Data Forwarding (Bypassing): Forward the result from the output of the EX or MEM stage directly to the input of the EX stage of the dependent instruction — without waiting for WB. This is the preferred hardware solution.

WAW (Write After Write) — Output Dependency

Two instructions both write to the same register. If I_2 completes before I_1 (possible in out-of-order processors), the final value in the register is incorrect.

- Solution: Ensure write-back order is maintained; use register renaming in out-of-order processors.

WAR (Write After Read) — Anti-Dependency

Instruction I_2 writes to a register that an earlier instruction I_1 still needs to read. In a simple in-order pipeline, WAR hazards rarely occur since reads precede writes. They become significant in out-of-order processors.

- Solution: Register renaming — allocate a physical register that is different from the original.

4.7.3 Control Hazards (Branch Hazards)

A control hazard occurs when the pipeline cannot determine which instruction to fetch next because a branch instruction is in progress. The branch outcome (taken or not taken) and the branch target address may not be known until the EX or MEM stage.

Example: In a 5-stage pipeline, a branch instruction enters EX at cycle 3. By then, instructions at cycle 4 (IF) and cycle 5 (IF+ID) have already been fetched based on the assumption of sequential execution. If the branch is taken, these two fetched instructions are wrong — 2 wasted cycles.

- Solution 1 — Stalling (Branch Delay Slot): Stall until the branch outcome is known. Penalty = number of stages until branch resolved.
- Solution 2 — Branch Prediction: Predict whether the branch will be taken or not, and fetch accordingly. If wrong, flush the incorrectly fetched instructions

(branch misprediction penalty). Modern processors achieve 95–99% prediction accuracy using dynamic branch predictors.

- Static Prediction: Always predict "not taken" (or "taken"). Simple but ~50–80% accurate depending on workload.
- Dynamic Prediction: Use a Branch Prediction Unit (BPU) that maintains a history of recent branch outcomes in a Branch History Table (BHT). Two-bit saturating counters provide good accuracy with minimal hardware.
- Solution 3 — Branch Delay Slot: The instruction immediately after the branch (the "delay slot") is always executed, regardless of the branch outcome. The compiler must fill this slot with a useful instruction or a NOP. Used in MIPS R2000/R3000.

🔍 Analysing Misconceptions: Data Forwarding Eliminates All Data Hazards

✗ MYTH: Data forwarding (bypassing) completely eliminates data hazards and no stalls are needed.

✓ REALITY: Data forwarding can eliminate most RAW hazards. However, load-use hazards — where a load instruction is immediately followed by an instruction that uses the loaded value — cannot be fully resolved by forwarding alone because the data is not available until after the MEM stage, while the dependent instruction needs it at the start of EX. A single-cycle stall (load-use hazard bubble) is unavoidable in most 5-stage pipelines even with forwarding.

 Let's Check

Q.No	Type	Question
1	MCQ	Which type of data hazard occurs when instruction I_2 reads a register before I_1 has written its result? (A) WAR (B) WAW (C) RAW (D) Structural
2	MCQ	A 5-stage pipeline processes 200 instructions with 1 ns per stage. What is the ideal pipelined execution time? (A) 1000 ns (B) 204 ns (C) 200 ns (D) 25 ns
3	Fill in Blank	A branch hazard is also known as a _____ hazard.
4	Fill in Blank	Inserting a wasted cycle between instructions to wait for data is called pipeline _____.
5	True/False	Data forwarding routes the result directly from EX/MEM stage output to the next instruction's EX stage input. (True/False)
6	True/False	A structural hazard can be eliminated by using separate instruction and data caches. (True/False)

Answers:

Q.No	Answer
1	(C) RAW (Read After Write) — the most common pipeline data hazard.
2	(B) 204 ns — $(k + n - 1) \times t = (5 + 200 - 1) \times 1 = 204$ ns.
3	Control

4	Stalling (bubble insertion)
5	True
6	True — separate I-cache and D-cache eliminate the memory structural hazard.

4.8 Impact of Pipelining on Instruction Set Design

Pipelining does not just affect hardware — it fundamentally shapes the design of the instruction set architecture (ISA). Processors designed with pipelining in mind (RISC architectures) make dramatically different design choices than those designed without it (early CISC architectures). This sub-unit examines the key principles that make an instruction set pipeline-friendly.

4.8.1 Fixed Instruction Length

Pipeline Stage 1 (IF) fetches instructions. If instructions have variable lengths, the processor cannot know the length of the current instruction until it partially decodes it — creating a dependency between IF and ID that breaks clean pipeline staging.

- RISC solution: All instructions are the same length (typically 32 bits). The IF stage can always fetch exactly one instruction per cycle without ambiguity.
- CISC challenge: x86 instructions range from 1 to 15 bytes. Modern x86 processors use complex pre-decode logic to determine instruction boundaries before they reach the main pipeline.

4.8.2 Load/Store Architecture

In a pipeline, the MEM stage can only perform one function per clock cycle. If arithmetic instructions also access memory (as in CISC), the EX and MEM stages become entangled, complicating the pipeline.

- RISC solution: Only LOAD and STORE instructions access memory. All arithmetic operates only on registers. This cleanly separates the EX and MEM pipeline stages.
- Example (RISC-V): LOAD R1, 100(R2) followed by ADD R3, R1, R4. Each uses its pipeline stage cleanly.

4.8.3 Three-Address Instruction Format

Instructions should explicitly specify both source registers and the destination register, enabling the pipeline to read two source operands in the ID stage and write the result in the WB stage without ambiguity.

- Example: ADD Rd, Rs1, Rs2 — three operands, all registers. Pipeline-friendly.
- Contrast: x86 ADD AX, BX — two operands, one is both source and destination. Requires more complex decode logic.

4.8.4 Delayed Branch and NOP Slots

To reduce branch penalty, RISC architectures like MIPS defined a "branch delay slot" — the instruction following a branch is always executed before the branch takes effect. Compilers fill this slot with a useful instruction (register move, load, or NOP) to reclaim one cycle that would otherwise be wasted.

4.8.5 RISC vs CISC in Context

Feature	RISC	CISC
Instruction Length	Fixed (32-bit)	Variable (1–15 bytes)
Memory Access	Load/Store only	Any instruction can access memory
Pipeline Design	Simple, clean stages	Complex, variable-length stages
Instruction Count	More instructions per program	Fewer instructions per program
Examples	ARM, RISC-V, MIPS, PowerPC	Intel x86, Motorola 68000

💡 Did You Know?

RISC-V, the open-source instruction set architecture developed at UC Berkeley, is gaining massive adoption in India. IIT Madras's Shakti project, C-DAC, and startups like InCore Semiconductors (Bengaluru) are building RISC-V processors for applications ranging from IoT sensors to automotive control units. The Government of India's National Mission on Interdisciplinary Cyber-Physical Systems (NM-ICPS) is partially funding RISC-V processor development as a strategic priority.

4.9 Parallel Computer Models

The fundamental limitation of a single processor is that it can execute only one instruction stream at a time (ignoring internal pipelining and superscalar execution). As problems grow in complexity — weather simulation, drug discovery, financial risk modelling, deep learning training — a single processor, no matter how fast, is insufficient. Parallel computing uses multiple processors working together to solve problems faster.

4.9.1 Flynn's Taxonomy

Michael Flynn's taxonomy (1966) classifies computer architectures based on the number of instruction streams and data streams they can handle simultaneously. It remains the most widely used classification framework.

Class	Full Name	Description & Examples
SISD	Single Instruction, Single Data	Classic von Neumann single-core processor. One instruction operates on one data element at a time. (Intel 8086, early Pentium)
SIMD	Single Instruction, Multiple Data	One instruction applies to multiple data elements simultaneously. Graphics processing, vector operations, multimedia. (Intel SSE/AVX, ARM NEON, GPUs)
MISD	Multiple Instruction, Single Data	Multiple processors execute different instructions on the same data. Theoretical; used in some fault-tolerant systems (NASA space shuttle flight computers).

MIMD	Multiple Instruction, Multiple Data	Multiple processors each execute different instructions on different data. Most common parallel model. (Multi-core CPUs, computer clusters, cloud computing)
-------------	--	--

4.9.2 Shared Memory Multiprocessors (MIMD — Tightly Coupled)

In a shared memory multiprocessor, multiple processors share a single global address space (a common, unified memory). Any processor can read or write any memory location directly using load and store instructions.

- Uniform Memory Access (UMA): All processors have equal access time to all memory locations. Simple to program but does not scale well beyond ~16 processors. Used in dual-socket servers (e.g., Intel Xeon multi-CPU workstations).
- Non-Uniform Memory Access (NUMA): Memory is physically distributed but logically shared. Access to a local memory bank is faster than access to a remote bank. Scales better — used in modern multi-socket servers and AMD EPYC processors.
- Programming Model: Standard multi-threaded programming (POSIX threads, OpenMP). Synchronisation is needed (mutexes, semaphores, barriers) to prevent race conditions.
- Cache Coherence: When multiple processors each have their own cache and share memory, a write by one processor must be reflected in the caches of all others. Cache coherence protocols (MESI, MOESI) maintain consistency.

4.9.3 Distributed Memory Multicomputers (MIMD — Loosely Coupled)

In a distributed memory system (multicomputer), each processor has its own private memory. Processors communicate by sending messages over an interconnection network — they cannot access each other's memory directly.

- No shared memory: Each node is an independent computer (processor + local memory).
- Communication: Message Passing Interface (MPI) is the dominant programming model. Processors explicitly send (MPI_Send) and receive (MPI_Recv) data packets over the interconnection network.
- Scalability: Scales to thousands or millions of processors. Used in supercomputers and cloud computing clusters.
- Interconnection Networks: The topology of the network (how nodes are connected) critically affects communication latency and bandwidth. Common topologies include: mesh, torus, hypercube, fat tree, and dragonfly.
- Examples: Beowulf clusters (Linux nodes + Gigabit Ethernet), Summit supercomputer (Oak Ridge, USA), Fugaku supercomputer (Japan), and PARAM Siddhi-AI (C-DAC, Pune, India).

💡 Did You Know?

C-DAC's PARAM Ananta supercomputer, commissioned in 2023 at IIT Gandhinagar, is built on distributed memory architecture with AMD EPYC processors and InfiniBand HDR interconnects (200 Gbps). It delivers 838 TeraFLOPS of peak performance and is being used for protein folding simulations relevant to the Indian pharmaceutical industry, including Cipla and Sun Pharma's drug discovery pipelines.

4.10 PRAM and VLSI Models

While shared-memory multiprocessors and multicomputers describe physical parallel architectures, computer scientists also use theoretical models to reason about the complexity of parallel algorithms — independently of specific hardware. The two most important theoretical models are PRAM (Parallel Random Access Machine) and the VLSI complexity model.

4.10.1 PRAM — Parallel Random Access Machine

The PRAM model is an idealised theoretical framework for designing and analysing parallel algorithms. It extends the classical RAM (Random Access Machine) sequential model to multiple processors.

PRAM Model Assumptions:

- There are p processors, all sharing a single global memory of unlimited size.
- All processors operate synchronously on a common clock.
- Each processor can access any memory location in one unit of time (no memory contention or latency).
- Processors can execute different instructions (MIMD model).

PRAM Variants — Concurrent vs Exclusive Access

The PRAM model has four variants based on how memory access conflicts are handled:

Variant	Description
EREW (Exclusive Read, Exclusive Write)	No two processors may read or write the same memory location simultaneously. Most restrictive; reflects realistic hardware constraints most closely.
CREW (Concurrent Read, Exclusive Write)	Multiple processors may read the same location simultaneously, but only one may write at a time. Used for broadcast operations.
CRCW (Concurrent Read, Concurrent Write)	Multiple processors may read AND write the same location simultaneously. Sub-variants: Common (all writers must write same value), Priority (highest-priority writer wins), Arbitrary.
ERCW (Exclusive Read, Concurrent Write)	Rarely used in practice; theoretical completeness of the taxonomy.

PRAM Complexity Analysis

Parallel algorithms are analysed using two measures:

- Time Complexity $T(n, p)$: The number of parallel steps required for n -sized input using p processors.
- Work $W(n, p) = T(n, p) \times p$: Total processor-time product. An efficient parallel algorithm should have $W \approx$ sequential algorithm complexity.

Example: Parallel sum of n numbers using $n/2$ processors (binary tree reduction):

- Step 1: All $n/2$ processors simultaneously add pairs: $(a_1+a_2), (a_3+a_4), \dots \rightarrow n/2$ partial sums.

- Step 2: $n/4$ processors add the partial sums $\rightarrow n/4$ results.
- After $\log_2 n$ steps: Final sum is computed.
- Time: $T = O(\log n)$; Work: $W = O(n \log n)$. Compared to sequential sum $O(n)$, work is slightly worse but time is exponentially better.

4.10.2 VLSI Model — Area-Time Complexity

The VLSI (Very Large Scale Integration) model addresses a fundamental question: given a computation, what is the minimum physical chip area (A) and time (T) required? This theoretical model is used to establish lower bounds on the resources needed to implement algorithms in silicon.

The key insight of the VLSI model is that both area and time are limited resources:

- Area A : The physical silicon area required for transistors, wires, and routing. Wires dominate area in modern chips — a key insight from the VLSI model.
- Time T : The number of clock cycles required to complete the computation.
- Area-Time Trade-off: For many computations, there is a fundamental trade-off: $A \times T^2 \geq \Omega(n^2)$. This means you can spend more area (more hardware) to reduce time, but not without limit — there is a lower bound on $A \times T^2$.

Practical Significance

The VLSI model explains why:

- Wider buses cost more silicon area but reduce communication time.
- Adding more pipeline stages (time) does not always require proportionally more area.
- Sorting networks (parallel comparator arrays) have area proportional to $n \log n$ — matching the optimal sequential sort in work complexity.
- The model influenced the design of systolic arrays — regular, rhythmic processor arrays used in matrix multiplication chips and modern AI accelerators (like Google TPU, used in Google Translate and Search).

Q Analysing Misconceptions: Parallel Processing and Linear Speedup

✗ MYTH: Using 10 processors will always make a program run 10 times faster (linear speedup).

✓ REALITY: Amdahl's Law states that speedup is fundamentally limited by the sequential fraction of a program. If 20% of a program is sequential (cannot be parallelised), then even with infinite processors the maximum speedup is $5\times (1 / 0.20)$. Additionally, communication overhead, load imbalance, and memory contention further reduce real speedup. True linear speedup is extremely rare; typical well-parallelised applications achieve 60–80% parallel efficiency.

Case Study

■ Case Study: Pipelining and Parallel Architecture in Google's Tensor Processing Unit (TPU)

Background

By 2013, Google's engineers projected that if users ran voice search with Deep Neural Networks (DNNs) for just 3 minutes per day, Google would need to double its entire data centre capacity just to serve inference requests. Existing CPUs and GPUs were adequate for model training but too slow and too power-hungry for serving millions of real-time inference requests. Google began designing a custom chip tailored for neural network inference — the Tensor Processing Unit (TPU v1), deployed in 2015.

The Problem

Neural network inference is dominated by matrix multiplication (multiply-accumulate operations). A conventional CPU executes these sequentially using its ALU. A GPU improves throughput using SIMD (Flynn taxonomy) but has general-purpose overhead. Google needed a chip where thousands of multiply-accumulate operations could execute every clock cycle with minimal overhead — achieving peak utilisation of arithmetic hardware.

Architecture Solution

- **Systolic Array** (inspired by VLSI model): TPU v1 contains a 256×256 matrix of multiply-accumulate (MAC) units — 65,536 MACs — arranged as a systolic array. Data flows rhythmically through the array (like heartbeats), with each MAC unit passing its partial result to its neighbour. This eliminates the need for a control bus, dramatically reducing chip area and power.
- **SIMD Parallel Processing**: All 65,536 MACs execute the same operation (multiply-accumulate) on different data simultaneously — a direct implementation of Flynn's SIMD model, extending it to a 2D array.

- Pipeline Stages: The data flow through the systolic array is itself pipelined — different rows of the matrix are processed in different pipeline stages simultaneously, achieving near-100% hardware utilisation.
- CISC-like control unit: The TPU uses a simple microprogrammed control unit to manage the small set of high-level instructions (MatrixMultiply, Activate, Read/Write to/from DRAM), demonstrating that microprogrammed control is valuable for domain-specific architectures.

Results

- TPU v1 achieved 15–30× higher performance-per-watt compared to contemporary CPUs and GPUs for inference workloads (as of 2015).
- TPU v4 (2021) — TPU pods of 4,096 chips connected in a 3D torus network form MIMD distributed-memory multicomputers; each pod delivers 1.1 ExaFLOPS of computing power.
- Indian connection: Google Cloud TPUs are used by IIT Bombay, IIT Delhi, and Tata Consultancy Services (TCS) Research for large language model training under Google's TPU Research Cloud programme.

Reference: Jouppi, N. P., et al. (2017). "In-Datacenter Performance Analysis of a Tensor Processing Unit." ISCA 2017, ACM/IEEE. Google AI Blog (2021). "TPU v4: Enabling Large-Scale AI."

Summary

Module 4 explored how instructions are processed within a CPU and how multiple processors can be organised to work in parallel, covering the full spectrum from individual transistors to planet-scale computing clusters.

Processing Unit Design and Instruction Execution:

- The CPU consists of the ALU, control unit, register file, PC, IR, MAR, and MDR — each playing a specific role in instruction processing.
- Every instruction passes through five stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB).
- Bus organisation — single, dual, or three-bus — determines how many data transfers can occur simultaneously within the CPU.

Control Unit Design:

- Hardwired control implements the control unit as a fixed logic circuit — fast but inflexible. Preferred for RISC architectures.
- Microprogrammed control stores control signals as data in a control store — flexible and easier to design for complex ISAs (CISC), but slower than hardwired.
- Modern processors often use a hybrid approach: hardwired for common instructions, microprogrammed for rare/complex ones.

Pipelining:

- Pipelining overlaps the execution of multiple instructions, dramatically increasing throughput toward one instruction per clock cycle.
- Speedup approaches the number of pipeline stages (k) for large instruction streams.

- Pipeline hazards — structural, data (RAW/WAW/WAR), and control — can reduce throughput. Solutions include stalling, data forwarding, and branch prediction.
- RISC ISA design (fixed-length instructions, load/store architecture) is driven by the need to build clean, efficient pipelines.

Parallel Architectures:

- Flynn's taxonomy classifies architectures as SISD, SIMD, MISD, or MIMD based on instruction and data stream counts.
- Shared-memory multiprocessors (UMA/NUMA) offer ease of programming but limited scalability.
- Distributed-memory multicomputers scale to thousands of nodes using message passing (MPI) over high-speed interconnects.
- PRAM provides a theoretical framework for parallel algorithm analysis. VLSI model establishes area-time trade-offs for silicon implementation.
- Amdahl's Law limits practical speedup based on the sequential fraction of a program.

Glossary

Term	Definition
Program Counter (PC)	A special-purpose register holding the memory address of the next instruction to be fetched.
Pipeline Hazard	A condition that prevents the next instruction from executing in its designated pipeline stage, causing a stall (bubble).
RAW Hazard	Read After Write — a data hazard where an instruction tries to read a register before a preceding instruction has written its result.
Data Forwarding	A hardware technique that routes the result of an instruction directly from the pipeline register to the input of a subsequent instruction, bypassing the write-back stage.
Branch Prediction	A processor mechanism that predicts whether a branch will be taken or not, allowing the pipeline to continue fetching instructions without stalling.
Microprogramming	A control unit design technique where machine instructions are implemented as sequences of microinstructions stored in a control memory.
Flynn's Taxonomy	Michael Flynn's 1966 classification of computer architectures into SISD, SIMD, MISD, and MIMD based on the number of instruction and data streams.
NUMA	Non-Uniform Memory Access — a shared-memory multiprocessor design where memory access time varies

	depending on which processor accesses which memory bank.
PRAM	Parallel Random Access Machine — a theoretical model for parallel computation with p synchronised processors sharing a global memory, used for parallel algorithm analysis.
Amdahl's Law	A law stating that the maximum parallel speedup of a program is limited by its sequential fraction: $\text{Speedup} \leq 1 / (\text{sequential fraction})$.

Self Assessment Questions

Section A: Multiple Choice Questions (MCQ)

1. Which register holds the instruction currently being executed by the CPU?
 - A) PC (Program Counter)
 - B) MAR (Memory Address Register)
 - C) IR (Instruction Register)
 - D) MDR (Memory Data Register)
2. In the instruction execution cycle, the Instruction Fetch (IF) stage reads the instruction from:
 - A) The register file
 - B) The ALU output
 - C) The memory location pointed to by the PC
 - D) The control store
3. Which bus organisation allows both source operands and the destination of an ALU operation to be transferred simultaneously in one clock cycle?
 - A) Single bus
 - B) Two-bus
 - C) Three-bus
 - D) System bus
4. In a hardwired control unit, control signals are generated by:
 - A) Fetching microinstructions from a control store
 - B) Fixed combinational/sequential logic circuits
 - C) Software running on a co-processor
 - D) A lookup table in main memory

-
5. Microprogrammed control is preferred over hardwired control when:
- A) Maximum speed is required
 - B) The instruction set is small and fixed
 - C) The instruction set is large and complex
 - D) Power consumption must be minimised
6. A 4-stage pipeline processes 50 instructions. Each stage takes 2 ns. What is the total pipelined execution time?
- A) 400 ns
 - B) 106 ns
 - C) 100 ns
 - D) 53 ns
7. What is the maximum theoretical speedup of a 6-stage pipeline over sequential execution (for a large number of instructions)?
- A) 2×
 - B) 3×
 - C) 6×
 - D) 12×
8. A structural hazard in a pipeline occurs when:
- A) An instruction reads a register before a previous instruction writes it
 - B) Two instructions in different pipeline stages require the same hardware resource
 - C) A branch instruction causes the wrong instruction to be fetched
 - D) An overflow occurs in the ALU
9. Data forwarding (bypassing) is used to resolve which type of hazard?
- A) Structural hazard
 - B) Control (branch) hazard
 - C) RAW (Read After Write) data hazard
-

- D) WAR (Write After Read) hazard
10. Which of the following cannot be resolved by data forwarding alone and requires a pipeline stall?
- A) ADD R1, R2, R3 followed by SUB R4, R1, R5
 - B) A load instruction immediately followed by an instruction using the loaded value
 - C) Two consecutive ADD instructions with different registers
 - D) A branch instruction
11. RISC architectures use fixed-length instructions primarily to:
- A) Reduce the number of instructions needed
 - B) Simplify the Instruction Fetch stage and enable clean pipelining
 - C) Increase the number of addressing modes
 - D) Support variable-length operands
12. In a pipeline, which of the following is NOT reduced by pipelining?
- A) Throughput time for many instructions
 - B) Average time per instruction (for large workloads)
 - C) Latency of a single individual instruction
 - D) Clock cycles per program (for large programs)
13. According to Flynn's taxonomy, a GPU performing the same shader operation on millions of pixels simultaneously is classified as:
- A) SISD
 - B) MIMD
 - C) MISD
 - D) SIMD
14. In a distributed memory multicomputer, processors communicate using:
- A) Shared global memory with cache coherence protocols
 - B) Direct memory access (DMA) to each other's memory
 - C) Message passing over an interconnection network

- D) A common system bus
15. NUMA (Non-Uniform Memory Access) refers to a system where:
- A) All processors have equal access time to all memory
 - B) Memory access time varies depending on which processor accesses which memory bank
 - C) Each processor has completely private memory with no sharing
 - D) Memory is distributed only on disk
16. The PRAM model assumes all processors operate:
- A) Asynchronously on independent clocks
 - B) Synchronously on a common clock
 - C) With message passing over a network
 - D) On separate private memories only
17. In EREW PRAM, which of the following is NOT permitted?
- A) Two processors reading different memory locations
 - B) Two processors writing different memory locations
 - C) Two processors reading the same memory location simultaneously
 - D) One processor reading and one processor writing different locations
18. Amdahl's Law states that parallel speedup is fundamentally limited by:
- A) The number of processors available
 - B) The speed of the interconnection network
 - C) The sequential fraction of the program that cannot be parallelised
 - D) The size of the shared cache
19. The VLSI model establishes that for many computations, there is a trade-off expressed as:
- A) $A + T \geq \Omega(n)$
 - B) $A \times T \geq \Omega(n)$
 - C) $A \times T^2 \geq \Omega(n^2)$

D) $A^2 \times T \geq \Omega(n^2)$

20. Google's Tensor Processing Unit (TPU) systolic array design is most directly inspired by which theoretical model?

- A) PRAM model
- B) Flynn's MIMD taxonomy
- C) VLSI area-time model and SIMD parallel processing
- D) Microprogrammed control theory

Section B: True / False

State whether the following statements are TRUE or FALSE.

1. In microprogrammed control, the control address register (CAR) is analogous to the Program Counter (PC) in normal instruction execution.
2. A deeper pipeline (more stages) always results in higher performance because it allows higher clock frequencies.
3. WAR (Write After Read) hazards are common in simple in-order pipelines.
4. In a MIMD architecture, each processor can execute a different instruction on different data simultaneously.
5. PRAM's CRCW variant allows multiple processors to write to the same memory location at the same time.

Section C: Short Answer Questions

Answer each question in 3–5 sentences.

1. Explain the difference between hardwired and microprogrammed control units. In which type of processor architecture is each more commonly used?
2. What is a RAW (Read After Write) hazard? Describe how data forwarding resolves this hazard with an example.
3. A 6-stage pipeline runs 1000 instructions with a 2 ns clock period. Calculate: (a) sequential execution time, (b) pipelined execution time, (c) speedup achieved.
4. Compare shared-memory multiprocessors and distributed-memory multicomputers. What are the key trade-offs between the two models?
5. State and explain Amdahl's Law. If 30% of a program is inherently sequential, what is the maximum speedup achievable with any number of processors?

Self Assessment: Answer Keys

MCQ Answer Key

Q.No	Correct Answer & Explanation
1	C) IR (Instruction Register) — holds the instruction fetched from memory that is currently being decoded and executed.
2	C) The memory location pointed to by the PC — the PC contains the address; the IF stage reads from that address.
3	C) Three-bus — Bus A carries Rs1, Bus B carries Rs2 to ALU, Bus C carries result to Rd — all in one cycle.
4	B) Fixed combinational/sequential logic circuits — hardwired control generates signals directly through gates, not from memory.
5	C) The instruction set is large and complex — microprogramming simplifies design and update of complex ISAs like x86.
6	B) 106 ns — $(k + n - 1) \times t = (4 + 50 - 1) \times 2 = 53 \times 2 = 106$ ns.
7	C) 6× — For large n, speedup approaches k (number of stages). A 6-stage pipeline approaches 6× throughput.
8	B) Two instructions require the same hardware resource simultaneously — e.g., both need memory access at the same cycle.
9	C) RAW data hazard — forwarding routes the computed value directly to the stage that needs it, avoiding a stall.
10	B) Load-use hazard — the loaded value is not available until after MEM stage; a one-cycle stall is unavoidable even with forwarding.
11	B) Simplify the Instruction Fetch stage and enable clean pipelining — fixed-length instructions are fully fetched in one cycle.

12	C) Latency of a single individual instruction — pipelining improves throughput but does not reduce per-instruction latency (still k stages).
13	D) SIMD — same operation (shader) applied to millions of data elements (pixels) simultaneously.
14	C) Message passing over an interconnection network — MPI is the standard programming model for distributed memory systems.
15	B) Memory access time varies depending on which processor accesses which memory bank — local memory is faster than remote.
16	B) Synchronously on a common clock — PRAM assumes perfect synchronisation; all processors execute one step per clock tick.
17	C) Two processors reading the same memory location simultaneously — EREW means Exclusive Read AND Exclusive Write.
18	C) The sequential fraction of the program — $\text{Speedup} \leq 1 / (\text{sequential fraction})$. Even infinite processors cannot parallelise sequential code.
19	C) $A \times T^2 \geq \Omega(n^2)$ — This area-time trade-off is the fundamental result of the VLSI complexity model for many computations.
20	C) VLSI area-time model and SIMD parallel processing — the systolic array is a direct physical realisation of VLSI-model rhythmic data flow with SIMD-style parallel MAC operations.

True / False Answer Key

Q.No	Answer / Explanation
1	TRUE — The CAR holds the address of the current microinstruction in the control store, serving the same purpose as the PC in normal instruction execution.
2	FALSE — Deeper pipelines allow higher clock frequencies, but they also increase branch misprediction penalty (more stages to flush) and load-use stall penalty. Beyond an optimal depth, the net performance can decrease. Intel Pentium 4 Prescott (31 stages) demonstrated this trade-off.
3	FALSE — WAR hazards are rare in simple in-order pipelines because reads (ID stage) precede writes (WB stage) in normal instruction flow. WAR hazards primarily occur in out-of-order processors where the instruction execution order differs from the program order.
4	TRUE — MIMD (Multiple Instruction, Multiple Data) allows each processor to execute an independent instruction on independent data — the basis of all multi-core processors and computing clusters.
5	TRUE — CRCW PRAM allows Concurrent Read and Concurrent Write. Sub-variants (Common, Priority, Arbitrary) define how write conflicts are resolved.

Short Answer Key

Q.No	Model Answer
1	Hardwired control implements control logic as fixed logic gates — very fast but rigid (changing the instruction set requires redesigning the hardware). It is used in RISC processors (ARM, RISC-V). Microprogrammed control stores microinstructions in a control memory — slower due to memory lookup but flexible (new instructions added by updating control store content). Used in CISC processors (Intel x86, IBM mainframes). Modern x86 processors use a hybrid: hardwired fast path + microprogrammed slow path.
2	RAW (Read After Write): instruction I_2 reads a register that I_1 hasn't written yet. Example: ADD R1, R2, R3 followed by SUB R4, R1, R5. I_2 needs R1 at its EX stage, but I_1 writes R1 only at WB. Data forwarding routes I_1 's result from the output of the EX pipeline register directly to the EX stage input of I_2 , bypassing the write-back stage. This eliminates the stall in most cases (except load-use hazard).
3	(a) Sequential: $n \times k \times t = 1000 \times 6 \times 2 = 12,000$ ns. (b) Pipelined: $(k + n - 1) \times t = (6 + 1000 - 1) \times 2 = 1005 \times 2 = 2,010$ ns. (c) Speedup = $12,000 / 2,010 \approx 5.97\times$ (approaching $6\times$ for large n , confirming the theoretical maximum speedup equals the number of pipeline stages).
4	Shared-memory multiprocessors share a global address space — easy to program (standard threading) but limited scalability due to memory bus contention and cache coherence overhead (typically <64 processors). Distributed-memory multicomputers give each processor private memory — scales to millions of nodes (supercomputers), but requires explicit message passing (MPI), making programming more complex. The key trade-

	off: shared memory is easier to program; distributed memory is more scalable.
5	Amdahl's Law: $\text{Speedup} = 1 / (s + (1-s)/p)$, where s = sequential fraction, p = number of processors. As $p \rightarrow \infty$: Maximum Speedup = $1/s$. If $s = 0.30$ (30% sequential): Maximum Speedup = $1/0.30 \approx 3.33\times$. No matter how many processors are added, the program cannot run more than $3.33\times$ faster. This law fundamentally limits the benefit of parallelism for programs with significant sequential sections.

ONLINE DEGREE PROGRAMS OFFERED

B.Com | BBA | MBA | MCA | M.Com | MA

